

Network Slicing in SDN con Mininet e Ryu

Network and Cloud Infrastructures — a.a. 2025/2026

Malafrente Alessio (DE9000200) | Persico Carmine (DE9000135) | Iannuzzi Corrado (DE9000026)

1 Obiettivo, ambiente e struttura del codice

Il progetto implementa il *network slicing* in ambiente SDN tramite un unico controller progettato per operare in due modalità distinte: **topology** e **service**.

Queste due logiche non possono essere attive contemporaneamente perché hanno scopi opposti:

- Il **topology slicing** punta all'isolamento: permette la comunicazione *solo* alle coppie di host autorizzate, assegnando a ciascuna un percorso fisico separato.
- Il **service slicing** punta alla differenziazione: richiede che *tutti* gli host possano parlarsi, ma instrada i pacchetti su percorsi diversi in base al tipo di servizio trasportato.

Al service slicing si affianca una **politica dinamica**, attivabile separatamente. Il traffico video non è sempre presente: quando è assente o trascurabile lo slice premium resterebbe inutilizzato, e il controller lo presta a una coppia di host best effort; il prestito viene revocato quando il carico video misurato supera una soglia (sezione 6). Il controller misura inoltre il carico dei due percorsi e lo espone a una dashboard.

Per gestire tutti gli scenari senza duplicare il codice, il comportamento desiderato viene selezionato all'avvio tramite variabili d'ambiente:

```
ryu-manager slicing_controller.py (topology, default)
SLICING_MODE=service ryu-manager slicing_controller.py (service)
DYNAMIC=1 SLICING_MODE=service ryu-manager slicing_controller.py (service + dinamico)
```

2 Topologia

Quattro switch e quattro host, con due percorsi paralleli fra le due zone della rete. I nodi **s1** e **s4** sono switch di accesso, **s2** e **s3** di transito.

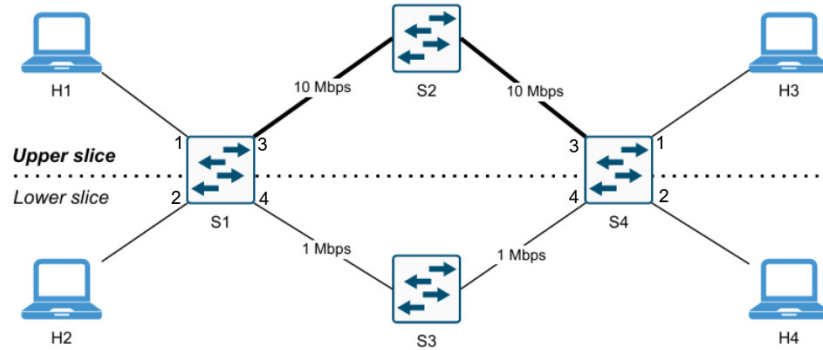


Figura 1: Topologia: due percorsi paralleli fra s1 e s4, con porte, banda e ritardo

Collegamento	Banda	Ritardo	Ruolo
host ↔ switch di accesso	non limitata	0,1 ms	non deve essere il collo di bottiglia
s1-s2-s4 (slice UPPER)	10 Mbps	2 ms	slice premium
s1-s3-s4 (slice LOWER)	1 Mbps	5 ms	slice best effort

I numeri di porta sono fissati esplicitamente in `addLink` e non lasciati alla numerazione implicita di Mininet, perché il controller li usa come costanti: riordinando le chiamate la numerazione cambierebbe e l'inoltro avverrebbe su porte errate *senza generare alcun errore*. Su entrambi gli switch di accesso vale la convenzione **porta 3 = upper**, **porta 4 = lower**. I link di accesso non hanno limite di banda proprio per non introdurre un secondo collo di bottiglia. I MAC sono statici perché il controller decide su `eth_src` ed `eth_dst`; IPv6 viene disattivato sui nodi, altrimenti il traffico multicast automatico di Linux genererebbe `packet_in` continui con sorgenti sconosciute.

3 Architettura del controller

Il controller è interamente **dichiarativo**: i percorsi non vengono appresi, sono definiti in tabelle consultate a ogni `packet_in`. La struttura centrale della modalità `topology` associa a ogni slice, per ogni switch, la porta d'uscita corrispondente a ciascuna porta d'ingresso:

```
SLICE_PATH = {
    'UPPER': {1: {1: 3, 3: 1}, # s1: h1 <-> s2
              2: {1: 2, 2: 1}, # s2: s1 <-> s4
              4: {1: 3, 3: 1}}, # s4: h3 <-> s2
```

```
}
  'LOWER': {1: {2: 4, 4: 2}, 3: {1: 2, 2: 1}, 4: {2: 4, 4: 2}},
}
```

La mappa e' simmetrica, quindi una sola definizione copre entrambi i versi di marcia. Accanto ad essa, `SLICE_OF_MAC` esprime l'appartenenza degli host agli slice, `HOST_LOCATION` la loro posizione, e `BCAST_TREE` l'albero di inoltro del traffico broadcast in modalita' service.

La scelta dichiarativa nasce dalla topologia ad anello. Un approccio basato sul MAC learning apprende inoltrando in *flooding* i MAC sconosciuti, e con un loop fisico questo genererebbe un *broadcast storm*. Definendo a priori le regole di inoltro il ciclo viene aggirato per costruzione: la logica di instradamento previene i loop logici, sostituendo l'intervento dello Spanning Tree Protocol.

Priorita' e scarto. OpenFlow risolve i match multipli scegliendo la regola a priorita' maggiore: 20 per il traffico video (match su livello 2, 3 e 4), 15 per la deviazione dinamica, 10 per l'inoltro ordinario, 5 per lo scarto, 0 per la table-miss che invia il pacchetto al controller. Lo scarto non usa un'azione dedicata: una flow entry *priva di istruzioni* fa scartare il pacchetto.

3.1 Il monitor e la dashboard

Dopo l'installazione di una regola i pacchetti successivi sono commutati dallo switch e non raggiungono piu' il controller: il controller conosce le proprie decisioni, non lo stato della rete. L'informazione si recupera dai contatori di OpenFlow, che vanno pero' richiesti esplicitamente. Un thread (`hub.spawn`) interroga ogni secondo `s1`, che e' un estremo di entrambi i percorsi core e vede quindi tutto il traffico che attraversa la rete, con `OFPPortStatsRequest` e `OFPPFlowStatsRequest`. Poiche' i contatori sono cumulativi, il rate si ottiene come differenza fra due campioni divisa per l'intervallo, misurato con il `duration` riportato dallo switch per non includere il ritardo di consegna della risposta. I campioni finiscono in `stats.json` con scrittura atomica, e una pagina statica (`dashboard.html`) li rilegge ogni secondo mostrando l'utilizzo dei due link core. Lo stesso sensore alimenta la politica dinamica della sezione 6.

4 Topology slicing: risultati

Configurazione: h1 comunica solo con h3 attraverso lo slice upper, h2 solo con h4 attraverso il lower.

```
mininet> net
h1 h1-eth0:s1-eth1
h2 h2-eth0:s1-eth2
h3 h3-eth0:s4-eth1
h4 h4-eth0:s4-eth2
s1 lo: s1-eth1:h1-eth0 s1-eth2:h2-eth0 s1-eth3:s2-eth1 s1-eth4:s3-eth1
s2 lo: s2-eth1:s1-eth3 s2-eth2:s4-eth3
s3 lo: s3-eth1:s1-eth4 s3-eth2:s4-eth4
s4 lo: s4-eth1:h3-eth0 s4-eth2:h4-eth0 s4-eth3:s2-eth2 s4-eth4:s3-eth2
c0
mininet> link
invalid number of args: link end1 end2 [up down]
mininet> links
h1-eth0->s1-eth1 (OK OK)
h2-eth0->s1-eth2 (OK OK)
h3-eth0->s4-eth1 (OK OK)
h4-eth0->s4-eth2 (OK OK)
s1-eth3->s2-eth1 (OK OK)
s2-eth2->s4-eth3 (OK OK)
s1-eth4->s3-eth1 (OK OK)
s3-eth2->s4-eth4 (OK OK)
mininet> ports
s1 lo:0 s1-eth1:1 s1-eth2:2 s1-eth3:3 s1-eth4:4
s2 lo:0 s2-eth1:1 s2-eth2:2
s3 lo:0 s3-eth1:1 s3-eth2:2
s4 lo:0 s4-eth1:1 s4-eth2:2 s4-eth3:3 s4-eth4:4
mininet> pingall
*** Ping: testing ping reachability
h1 -> X h3 X
h2 -> X X h4
h3 -> h1 X X
h4 -> X h2 X
*** Results: 66% dropped (4/12 received)
mininet> h1 ping -c5 h3
PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.
64 bytes from 10.0.0.3: icmp_seq=1 ttl=64 time=12.0 ms
64 bytes from 10.0.0.3: icmp_seq=2 ttl=64 time=13.6 ms
64 bytes from 10.0.0.3: icmp_seq=3 ttl=64 time=12.1 ms
64 bytes from 10.0.0.3: icmp_seq=4 ttl=64 time=11.9 ms
64 bytes from 10.0.0.3: icmp_seq=5 ttl=64 time=11.9 ms

--- 10.0.0.3 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4005ms
rtt min/avg/max/mdev = 11.857/12.154/12.953/0.405 ms
mininet> h1 ping -c5 h4
PING 10.0.0.4 (10.0.0.4) 56(84) bytes of data.
64 bytes from 10.0.0.4: icmp_seq=1 ttl=64 time=23.4 ms
64 bytes from 10.0.0.4: icmp_seq=2 ttl=64 time=23.6 ms
64 bytes from 10.0.0.4: icmp_seq=3 ttl=64 time=23.4 ms
64 bytes from 10.0.0.4: icmp_seq=4 ttl=64 time=24.0 ms
64 bytes from 10.0.0.4: icmp_seq=5 ttl=64 time=24.8 ms

--- 10.0.0.4 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4007ms
rtt min/avg/max/mdev = 23.378/23.844/24.768/0.522 ms
```

```
mininet> h1 arp -n
Address      HWtype  HWaddress      Flags Mask      Iface
10.0.0.2     (incomplete)
10.0.0.4     (incomplete)
10.0.0.3     ether    00:00:00:00:00:03 C               h1-eth0
mininet>
```

(a) Porte, isolamento e ritardi

(b) Cache ARP di h1: risolto solo il partner di slice

Figura 2: Verifica dell'isolamento. `pingall` restituisce 66% dropped (4/12).

La cache ARP di h1 mostra il punto piu' interessante: 10.0.0.3 e' risolto, mentre 10.0.0.2 e 10.0.0.4 restano (*incomplete*). L'isolamento agisce quindi **gia' a livello ARP** e non sul traffico IP: la ARP request di h1 segue il percorso dello slice upper e non raggiunge mai h2 e h4, dai quali non arriva quindi alcuna risposta.

Misura	Upper (h1–h3)	Lower (h2–h4)	Atteso
RTT medio (ping)	12,2 ms	23,8 ms	rapporto $\approx 1:2$
Throughput (iperf TCP)	9,55 Mbps	1,21 Mbps	rapporto $\approx 10:1$
Cattura sui link core	solo h1↔h3	solo h2↔h4	separazione totale

L'ultima riga e' stata verificata con due `tcpdump` simultanei su `s2-eth1` e `s3-eth1` durante due ping in parallelo: i timestamp si sovrappongono, quindi i due flussi coesistono sulla stessa rete e ciascuno usa esclusivamente il proprio link core. Le catture sono nel repository.

5 Service slicing: risultati

Tutti gli host si raggiungono (`pingall: 0% dropped`), ma il percorso dipende dal servizio: il traffico UDP verso la porta 9999, convenzionalmente utilizzata nel progetto per traffico video, usa lo slice upper da 10 Mbps, ogni altro traffico il lower da 1 Mbps.

Il test finale usa **la stessa coppia di host e lo stesso bitrate**, variando solo la porta UDP di destinazione:

Flusso h1 → h3	Slice	Ricevuto	Perdita	Jitter
UDP porta 9999 (video)	UPPER	9,72 Mbps	53%	0,43 ms
UDP porta 5001 (dati)	LOWER	0,97 Mbps	95%	1,64 ms

I due flussi sono identici in tutto tranne che nella porta di destinazione, eppure il primo riceve dieci volte la banda del secondo: e' la prova che il percorso viene scelto in base al servizio. Le perdite elevate sono volute. La sorgente offre circa 21 Mbps in UDP, che non rallenta quando la rete e' satura: cio' che eccede la capacita' del percorso viene scartato, cioe' circa meta' sul link da 10 Mbps e il 95% su quello da 1 Mbps. Saturare il percorso serve a misurarne la capacita': il rate ricevuto coincide infatti con la banda nominale dei due slice. Anche il jitter e' piu' alto sul lower, dove i pacchetti escono da una coda piena.

[SWITCH] s1 connesso	mininet> h3 iperf -s -u -p 9999 &
[DROP] s1: sorgente sconosciuta fa:e7:95:dc:bc:7b	mininet> h1 iperf -c 10.0.0.3 -u -p 9999 -b 20M -t 10 -i 1
[SWITCH] s2 connesso	-----
[SWITCH] s3 connesso	Client connecting to 10.0.0.3, UDP port 9999
[DROP] s2: sorgente sconosciuta 12:65:70:11:aa:38	Sending 1470 byte datagrams, IPG target: 560.76 us (kalman adjust)
[SWITCH] s4 connesso	UDP buffer size: 208 KByte (default)
[DROP] s3: sorgente sconosciuta 56:1b:f1:54:57:fb	-----
[DROP] s3: sorgente sconosciuta 8e:20:87:fe:0d:40	[1] Local 10.0.0.1 port 4043 connected with 10.0.0.3 port 9999
[DROP] s2: sorgente sconosciuta 46:1bc1:e5:34:44	[ID] Interval Transfer Bandwidth
[DROP] s4: sorgente sconosciuta 36:de:01:02:92:56	[1] 0.0000-1.0000 sec 2.50 MBytes 21.0 Mbits/sec
[FLOW] s4 [LOWER] 00:00:00:00:00:03 -> 00:00:00:00:00:01 : porta 1 -> 4	[1] 1.0000-2.0000 sec 2.50 MBytes 21.0 Mbits/sec
[FLOW] s3 [LOWER] 00:00:00:00:00:03 -> 00:00:00:00:00:01 : porta 2 -> 1	[1] 2.0000-3.0000 sec 2.50 MBytes 21.0 Mbits/sec
[FLOW] s1 [LOWER] 00:00:00:00:00:03 -> 00:00:00:00:00:01 : porta 4 -> 1	[1] 3.0000-4.0000 sec 2.50 MBytes 21.0 Mbits/sec
[FLOW] s1 [UPPER VIDEO] 00:00:00:00:00:01 -> 00:00:00:00:00:03 : porta 1 -> 3	[1] 4.0000-5.0000 sec 2.50 MBytes 21.0 Mbits/sec
[FLOW] s2 [UPPER VIDEO] 00:00:00:00:00:01 -> 00:00:00:00:00:03 : porta 1 -> 2	[1] 5.0000-6.0000 sec 2.50 MBytes 21.0 Mbits/sec
[FLOW] s4 [UPPER VIDEO] 00:00:00:00:00:01 -> 00:00:00:00:00:03 : porta 3 -> 1	[1] 6.0000-7.0000 sec 2.50 MBytes 21.0 Mbits/sec
[FLOW] s4 [LOWER] 00:00:00:00:00:03 -> 00:00:00:00:00:01 : porta 1 -> 4	[1] 7.0000-8.0000 sec 2.50 MBytes 21.0 Mbits/sec
[FLOW] s3 [LOWER] 00:00:00:00:00:03 -> 00:00:00:00:00:01 : porta 2 -> 1	[1] 8.0000-9.0000 sec 2.50 MBytes 21.0 Mbits/sec
[FLOW] s1 [LOWER] 00:00:00:00:00:03 -> 00:00:00:00:00:01 : porta 4 -> 1	[1] 9.0000-10.0000 sec 2.50 MBytes 21.0 Mbits/sec
[FLOW] s1 [LOWER] 00:00:00:00:00:01 -> 00:00:00:00:00:03 : porta 1 -> 4	[1] 0.0000-10.0003 sec 25.0 MBytes 21.0 Mbits/sec
[FLOW] s3 [LOWER] 00:00:00:00:00:01 -> 00:00:00:00:00:03 : porta 1 -> 2	[1] Sent 17836 datagrams
[FLOW] s4 [LOWER] 00:00:00:00:00:01 -> 00:00:00:00:00:03 : porta 4 -> 1	[1] Server Report:
[FLOW] s1 [LOWER] 00:00:00:00:00:01 -> 00:00:00:00:00:03 : porta 1 -> 4	[ID] Interval Transfer Bandwidth Jitter Lost/Total Datagrams
[FLOW] s3 [LOWER] 00:00:00:00:00:01 -> 00:00:00:00:00:03 : porta 1 -> 2	[1] 0.0000-10.0049 sec 11.7 MBytes 9.72 Mbits/sec 0.433 ms 9500/17836 (53%)
[FLOW] s4 [LOWER] 00:00:00:00:00:01 -> 00:00:00:00:00:03 : porta 4 -> 1	[1] 0.0000-10.0049 sec 50 datagrams received out-of-order
	mininet>

Figura 3: Log del controller in modalita' service e risultato del flusso video

Instradamento asimmetrico. Nei log il traffico di ritorno h3 → h1 viaggia sul lower anche durante il test video. Il controller sceglie lo slice guardando la porta UDP di *destinazione*: i pacchetti del video sono diretti alla 9999, mentre quelli che h3 rimanda a h1 sono diretti alla porta da cui il client ha trasmesso, diversa dalla 9999, e sono quindi trattati come traffico ordinario. Per uno streaming reale e' il comportamento desiderabile: nel verso di ritorno passano solo pochi messaggi di controllo, e solo il verso che trasporta il video richiede la banda dello slice premium.

Traffico realistico. `iperf` genera flussi a rate costante e non misura il ritardo. Le prove sono state ripetute con **D-ITG**, che consente di specificare separatamente dimensione dei pacchetti e cadenza, ciascuna con una distribuzione statistica, e riporta ritardo e jitter per flusso. Mantenendo invariata la coppia di host e variando solo la porta:

Flusso h1 → h3 (D-ITG)	Slice	Ritardo medio	Ritardo max	Perdita
1400 B a cadenza costante, porta 9999	UPPER	5,7 ms	9,6 ms	0%
64–1024 B a raffica, porta 5001	LOWER	455 ms	553 ms	36,5%

Il ritardo di 455 ms non e' propagazione, poiche' il percorso lower ne introduce 10 ms in tutto: e' **tempo di attesa in coda** davanti al link saturo. Il confronto con i 5,7 ms dell'upper fa capire la differenza fra i due slice: non cambia solo la banda, ma anche il ritardo e la perdita che un'applicazione subisce. Con `iperf` questo dato non sarebbe stato ottenibile.

6 Slicing dinamico

Il traffico video non e' sempre presente: esiste solo mentre una sorgente trasmette verso la porta 9999. Quando e' assente lo slice premium resta inutilizzato, mentre il traffico best effort si accontenta di 1 Mbps. La politica

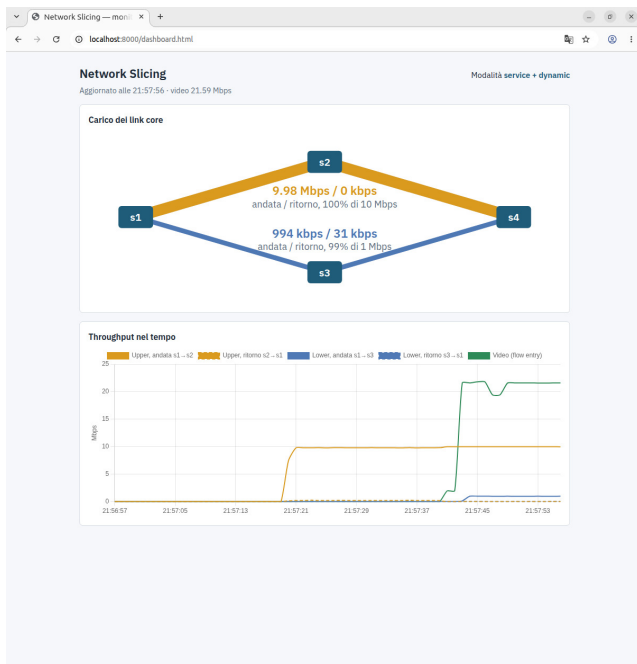
dinamica presta allora il percorso upper alla coppia $h2 \leftrightarrow h4$, in entrambi i versi, e lo revoca quando il carico video misurato supera una soglia. Ha senso solo in modalita' service, dove esiste la nozione di traffico video: richiesta in topology, viene segnalata e ignorata.

Il punto di misura. Il carico video e' misurato sui contatori delle *flow entry* che classificano il video, tramite una *OFPPFlowStatsRequest* filtrata su *udp_dst*, e non sui contatori del link upper. Sul link upper transita anche il traffico prestato, quindi la sua misura dipenderebbe dalle decisioni del controller stesso; quella sulle regole video ne e' indipendente. Va notato che la regola risiede sull'ingresso di *s1*, quindi misura il video *offerto*, a monte del collo di bottiglia: e' la grandezza corretta, perche' il controller deve conoscere la domanda di banda e non cio' che ne sopravvive.

Le due soglie. La decisione usa un limite basso (1 Mbps) e un limite alto (3 Mbps): il prestito viene concesso quando il traffico video scende sotto il primo e revocato quando supera il secondo. Fra i due limiti lo stato resta quello che era, qualunque esso sia. In questo modo le normali fluttuazioni del traffico video non provocano continui cambi di percorso: con un limite unico, ogni oscillazione della misura attorno a quel valore farebbe spostare il traffico da uno slice all'altro, con riordino dei pacchetti e riscrittura continua delle tabelle.

L'attuazione. Il prestito installa una regola a priorita' 15, con match sulla sola coppia di MAC e senza *idle_timeout*, su *s1* per il verso $h2 \rightarrow h4$ e su *s4* per quello opposto. La priorita' 15 sta fra l'inoltro ordinario (10) e il video (20): copre il primo senza mai toccare il secondo. Gli switch di core non vanno riprogrammati, perche' TRANSIT istrada in base alla porta d'ingresso e non allo slice. La revoca usa *OFPPC_DELETE_STRICT*, che agisce solo sulla regola avente esattamente quel match e quella priorita'.

Flusso $h2 \rightarrow h4$ (iperf TCP)	Percorso	Throughput
Video sotto 1 Mbps, prestito attivo	UPPER	9,54 Mbps
Video sopra 3 Mbps, prestito revocato	LOWER	1,21 Mbps



(a) Upper al 100%, lower al 98%

```

[FLOW] s1 [UPPER VIDEO] 00:00:00:00:00:01 -> 00:00:00:00:00:03 : porta 1 -> 3
[FLOW] s2 [UPPER VIDEO] 00:00:00:00:00:01 -> 00:00:00:00:00:03 : porta 1 -> 2
[FLOW] s4 [UPPER VIDEO] 00:00:00:00:00:01 -> 00:00:00:00:00:03 : porta 3 -> 1
[DYNAMIC] s1: rimossa la deviazione 00:00:00:00:00:02 -> 00:00:00:00:00:04
[DYNAMIC] s4: rimossa la deviazione 00:00:00:00:00:04 -> 00:00:00:00:00:02
[DYNAMIC] video a 21.61 Mbps: h2 <-> h4 torna sullo slice lower
[FLOW] s1 [LOWER] 00:00:00:00:00:02 -> 00:00:00:00:00:04 : porta 2 -> 4
[FLOW] s3 [LOWER] 00:00:00:00:00:02 -> 00:00:00:00:00:04 : porta 1 -> 2
[FLOW] s4 [LOWER] 00:00:00:00:00:02 -> 00:00:00:00:00:04 : porta 4 -> 2
[FLOW] s1 [LOWER] 00:00:00:00:00:04 -> 00:00:00:00:00:02 : porta 2 -> 4
[FLOW] s3 [LOWER] 00:00:00:00:00:04 -> 00:00:00:00:00:02 : porta 2 -> 1
[FLOW] s1 [LOWER] 00:00:00:00:00:04 -> 00:00:00:00:00:02 : porta 4 -> 2
[FLOW] s3 [LOWER] 00:00:00:00:00:04 -> 00:00:00:00:00:02 : porta 1 -> 4
[FLOW] s1 [LOWER] 00:00:00:00:00:01 -> 00:00:00:00:00:03 : porta 1 -> 4
[FLOW] s3 [LOWER] 00:00:00:00:00:01 -> 00:00:00:00:00:03 : porta 1 -> 2
[FLOW] s4 [LOWER] 00:00:00:00:00:01 -> 00:00:00:00:00:03 : porta 4 -> 1
[FLOW] s1 [LOWER] 00:00:00:00:00:03 -> 00:00:00:00:00:01 : porta 1 -> 4
[FLOW] s3 [LOWER] 00:00:00:00:00:03 -> 00:00:00:00:00:01 : porta 2 -> 1
[FLOW] s4 [LOWER] 00:00:00:00:00:03 -> 00:00:00:00:00:01 : porta 4 -> 1
[FLOW] s1 [LOWER] 00:00:00:00:00:03 -> 00:00:00:00:00:01 : porta 2 -> 1
[FLOW] s3 [LOWER] 00:00:00:00:00:03 -> 00:00:00:00:00:01 : porta 4 -> 1
[FLOW] s4 [LOWER] 00:00:00:00:00:03 -> 00:00:00:00:00:01 : porta 1 -> 4
[FLOW] s1 [LOWER] 00:00:00:00:00:01 -> 00:00:00:00:00:03 : porta 1 -> 4
[FLOW] s3 [LOWER] 00:00:00:00:00:01 -> 00:00:00:00:00:03 : porta 1 -> 2
[FLOW] s4 [LOWER] 00:00:00:00:00:01 -> 00:00:00:00:00:03 : porta 4 -> 1
[DYNAMIC] s1: 00:00:00:00:00:02 -> 00:00:00:00:00:04 deviato su porta 3
[DYNAMIC] s4: 00:00:00:00:00:04 -> 00:00:00:00:00:02 deviato su porta 3
[DYNAMIC] video a 0.00 Mbps: presto lo slice upper a h2 <-> h4

```

(b) Revoca e successivo nuovo prestito

Figura 4: Momento del rientro: il video occupa lo slice upper e il traffico best effort torna sul lower, dove satura il proprio link.

La prova e' stata ripetuta con una sorgente D-ITG a cadenza esponenziale, il cui rate non e' costante ma oscilla attorno a un valore medio. In venticinque secondi di traffico variabile il controller ha cambiato stato due sole volte: una revoca all'arrivo del video e un nuovo prestito alla sua fine. Le oscillazioni del rate non hanno quindi provocato cambi di percorso ripetuti, grazie alla distanza fra i due limiti. La revoca e' scattata a 4,28 Mbps, cioe' non appena la misura ha superato il limite alto di 3 Mbps.

Il prestito riguarda una sola coppia di host, scelta in anticipo. E' una semplificazione voluta: cosi' il comportamento e' facile da osservare nei test, e al controller basta un unico valore booleano per ricordare se il prestito e' attivo. Estendere il meccanismo a piu' coppie richiede di stabilire un criterio per scegliere a chi prestare lo slice.

7 Granularita' delle flow entry

Se le regole di inoltro si basassero unicamente sugli indirizzi MAC, un semplice *ping* iniziale genererebbe una regola cosi' generica da catturare tutto il traffico successivo fra i due host, streaming video compreso. I pacchetti

video non arriverebbero piu' al controller, la classificazione non verrebbe mai eseguita e il video finirebbe sullo slice sbagliato.

Ogni regola e' quindi **specifica quanto la classe di traffico che rappresenta**: se una classe si distingue per la porta UDP, il match deve arrivare fino alla porta UDP.

```
if ip4 is None:
    match = parser.OFPMatch(in_port=in_port, eth_src=src, eth_dst=dst,
                             eth_type=eth.ether_type)
elif ip4.proto == in_proto.IPPROTO_UDP:
    match = parser.OFPMatch(in_port=in_port, eth_src=src, eth_dst=dst,
                             eth_type=ether_types.ETH_TYPE_IP,
                             ip_proto=in_proto.IPPROTO_UDP,
                             udp_dst=udp_hdr.dst_port)
    priority = PRIO_VIDEO if is_video else PRIO_FORWARD
else:
    match = parser.OFPMatch(in_port=in_port, eth_src=src, eth_dst=dst,
                             eth_type=ether_types.ETH_TYPE_IP,
                             ip_proto=ip4.proto)
```

Per fare match su `udp_dst` OpenFlow richiede che siano specificati anche `eth_type` e `ip_proto`: i prerequisiti servono allo switch per sapere come interpretare i byte in cui cercare la porta.

```
ncis@ncis-vm:~/Desktop/ncis$ sudo ovs-ofctl dump-flows s1 -O OpenFlow13
cookie=0x0, duration=14.344s, table=0, n_packets=6262, n_bytes=9468144, idle_timeout=30, priority=20,udp,in_port="s1-eth1",dl_src=00:00:00:00:00:01,dl_dst=00:00:00:00:00:03,tp_dst=9999 actions=output:"s1-eth3"
cookie=0x0, duration=7.328s, table=0, n_packets=3134, n_bytes=4738608, idle_timeout=30, priority=10,udp,in_port="s1-eth1",dl_src=00:00:00:00:00:01,dl_dst=00:00:00:00:00:03,tp_dst=5001 actions=output:"s1-eth4"
cookie=0x0, duration=8.843s, table=0, n_packets=0, n_bytes=0, idle_timeout=30, priority=10,arp,in_port="s1-eth1",dl_src=00:00:00:00:00:00:01,dl_dst=00:00:00:00:00:03 actions=output:"s1-eth4"
cookie=0x0, duration=8.815s, table=0, n_packets=0, n_bytes=0, idle_timeout=30, priority=10,arp,in_port="s1-eth4",dl_src=00:00:00:00:00:00:03,dl_dst=00:00:00:00:00:01 actions=output:"s1-eth1"
cookie=0x0, duration=1066.247s, table=0, n_packets=62, n_bytes=13286, priority=0 actions=CONTROLLER:65535
```

Figura 5: `dump-flows` su `s1` con due flussi UDP attivi. Le prime due regole hanno identici `in_port`, `dl_src` e `dl_dst`: cambia solo `tp_dst` (9999 contro 5001), e con esso priorita' (20 contro 10) e porta di uscita (`s1-eth3` contro `s1-eth4`).

8 Il traffico broadcast e l'assenza di flooding

In modalita' topology lo slice di un pacchetto e' determinato dal MAC sorgente: il broadcast consulta `SLICE_PATH` come ogni altro pacchetto e segue il cammino predefinito, ignorando le interfacce esterne allo slice di origine. Ne discende la proprieta' piu' importante del meccanismo, cioe' la **totale assenza di flooding**: il pacchetto esce sempre da un'unica porta. Funziona perche' ogni slice collega due soli host lungo un percorso lineare, quindi raggiungere l'altro capo coincide con il seguire la direzione opposta a quella di arrivo.

Per il traffico di gruppo il controllo di isolamento sui MAC viene saltato, perche' un indirizzo come `ff:ff:ff:ff:ff:ff` non appartiene ad alcuno slice; i tre casi restano comunque esaustivi e disgiunti:

Destinazione	Classe	Trattamento
Broadcast o multicast	BUM	Segue il percorso dello slice del mittente
Unicast, stesso slice	unicast	Inoltro e installazione della flow entry
Unicast, altro slice	unicast	Regola di scarto (priorita' 5)

Il traffico di gruppo non genera alcuna flow entry: installare regole fisse per il broadcast occuperebbe spazio nelle tabelle per pochissimi pacchetti, dato che la cache ARP degli host memorizza le risoluzioni. In modalita' service, invece, il broadcast deve raggiungere tutti gli host e va replicato su piu' porte dello stesso switch: la mappa uno-a-uno {porta: porta} non basta, e si usa `BCAST_TREE`, del tipo {porta: [lista di porte]}, un albero statico che esclude `s2`.

9 Altre scelte di progetto ed esecuzione

Richieste duplicate al controller. In un controller reattivo la regola viene installata solo dopo l'arrivo del primo pacchetto di un flusso. Fra l'istante in cui il controller decide e quello in cui la regola diventa attiva sullo switch, pero', i pacchetti successivi dello stesso flusso sono gia' in viaggio e raggiungono anch'essi il controller: nei primi test, con un flusso da 20 Mbps, la stessa regola veniva installata cinque o sei volte per switch. E' un effetto noto del controllo reattivo, e la contromisura adottata e' semplice: il controller ricorda per 30 s le regole appena installate (`recent_flows`) e, quando riceve una richiesta gia' soddisfatta, si limita a restituire il pacchetto allo switch senza reinstallare nulla. Le installazioni sono scese a una per switch.

Misure in UDP. Le misure di riferimento del service slicing sono in UDP. In TCP il client `iperf` riporta i byte scritti nel buffer di invio del sistema operativo, non quelli arrivati a destinazione: i dati in eccesso si accumulano in coda, i valori misurati superano la banda nominale del link (1,21 Mbps su un link da 1 Mbps) e il test dura piu' del richiesto. In UDP questo accumulo non avviene e il Server Report riporta il rate effettivamente ricevuto.

Scadenza delle regole. Le flow entry hanno un `idle_timeout` di 30 s, che limita la crescita delle tabelle e fa si' che una modifica alla configurazione abbia effetto in tempi brevi senza riavviare il controller. Le regole della politica dinamica sono l'unica eccezione: non hanno scadenza, perche' devono durare quanto la condizione che le ha generate, ed e' il controller a rimuoverle.

Gli screenshot completi di tutti gli esperimenti, compresi i test TCP, le catture `tcpdump` e le verifiche di configurazione, sono nel repository sotto `docs/screenshots/`; il `README.md` riporta la procedura di installazione, i comandi di riproduzione e i vincoli sulle versioni.